System Architecture Document — Eco-Loop Building Agents

1. Overview

Eco-Loop Building Agents closes the loop between a live physics simulation and an LLM controller: **EnergyPlus** (via `pyenergyplus.api`, not the CLI) runs a continuous simulation of a 5-zone DOE reference office building. At a fixed cadence, it hands a compact state snapshot to a locally-hosted **Llama 3.1 8B** model (Ollama) through a real **MCP (Model Context Protocol)** tool server. The model calls tools to inspect state/forecast and commits new heating/cooling setpoints, which are written back into the *same running simulation* via EMS actuators — no restart, no human in the loop.

This document covers the four areas of the system most relevant to reviewing its engineering: tool-calling architecture, prompt engineering, latency management, and how it avoids feeding raw simulation output to the LLM.

---

2. Tool-Calling Architecture

The agent does not call Python functions dressed up as "tools" — it talks to a genuine MCP server over a real subprocess boundary.

- **Three tools**, defined in `src/mcp_server.py` with the official `mcp` SDK's `FastMCP`, spawned by `agent_client.py` as a child process over stdio and invoked through a real `ClientSession`:
  - `get_building_state()` — current time, outdoor weather, per-zone temperature/RH/PMV, whole-facility power draw, active setpoints.
  - `get_forecast(hours_ahead)` — outdoor temperature and synthetic grid carbon-intensity forecast, used to decide whether to pre-condition now for a lower-carbon hour later.
  - `set_zone_setpoints(cooling_c, heating_c, rationale)` — the only tool with a side effect; clamps to a safe band server-side and returns what was actually applied.
- **The state-visibility problem.** The MCP server is a separate OS process from the EnergyPlus driver and has no direct access to its memory. Rather than build bidirectional IPC, the driver writes the latest snapshot

to `logs/live_state.json` immediately before invoking the agent, and the server's read tools simply load whatever is there. The server stays a genuine, independently-callable tool boundary (it could serve any live building, or a different client entirely); only the read path uses this file handoff — the write path (`set_zone_setpoints`) returns its result through the normal MCP `CallToolResult`, no file involved.

- **Agent loop** (`AgentClient._decide_async`): a system prompt plus a pre-filled, compact state summary in the first user turn — so the model isn't forced to spend a round calling `get_building_state` for values it's already been given. It may call `get_forecast` if it wants more information, capped at **3 tool-calling rounds**. The loop terminates the instant `set_zone_setpoints` is called; any other response triggers a "call `set_zone_setpoints` now" nudge on the next round, so every decision cycle ends in a committed action, not a dangling conversation.
- **Sync simulation, async protocol.** EnergyPlus's EMS callbacks are synchronous ctypes callbacks; the MCP SDK is asyncio-based. A single background thread runs one persistent event loop for the life of the whole run — the MCP server subprocess is spawned **once** (not once per decision; at ~49 decisions per run that would mean 49 subprocess spawns for no benefit), and each `decide()` call is submitted to that loop via `run_coroutine_threadsafe`, bridging the simulation's synchronous callback world with the agent's async tool-calling world without either side blocking the other.

---

3. Prompt Engineering Strategies

Rule-based system prompt, not open-ended. The system prompt states the objective (minimize energy/carbon while keeping occupants comfortable), the occupied window (06:00–22:00), a target PMV band (±0.5, roughly 21–25°C), and permission to set back aggressively when unoccupied (cooling ~28°C, heating ~16°C). Giving the model a bounded decision space rather than an open-ended goal made its outputs far more consistent across runs.

Concrete, numeric pre-conditioning instruction. The building's thermal mass takes 2–4 hours to recover from a deep overnight setback. An early version of the prompt only said this in general terms, and the agent regularly started warming the building at occupancy start (06:00) instead of before it — occupied-hours comfort compliance measured only 79.4%. Rewriting the instruction to name the exact hour and reason ("if the current hour is 04:00 or 05:00, you MUST stop setting back and move toward the comfort band *now*, even though nobody is there yet") recovered most of the gap (79.4% → 90.9%) at essentially the same energy savings. This is the clearest evidence in the project that prompt wording measurably changes closed-loop physical outcomes, not just chat quality — and that vague instructions to a small (8B) model need to be replaced with concrete, numbered ones, verified against the decision log rather than assumed.

Fixed-format user turn. Rather than free text, every decision turn presents the same structured snapshot: time + occupancy tag, outdoor temperature, per-zone temperature/PMV/RH dictionaries, facility power draw, current setpoints, and carbon intensity. Consistent structure lets an 8B model reason reliably turn after turn instead of re-parsing varying prose.

Never trust raw model output. `llama3.1:8b`'s tool-calling returns argument values with inconsistent types (e.g., numeric setpoints as strings) and, observed during testing, occasionally an invalid ordering (heating above cooling). Every tool argument is coerced to the correct type defensively (`_coerce_args` in `agent_client.py`), and setpoints are clamped and deadband-checked in **two independent places** — inside the `set_zone_setpoints` MCP tool (`src/safety.py`), and again as a hard floor in `ep_runner.py` immediately before the value reaches an EnergyPlus actuator. Prompt instructions describe the desired behavior; validation code

guarantees it.

---

4. Prompt Latency Management

A locally-hosted 8B model has real, variable latency: **~20–90 seconds per round-trip** once warm, and roughly a one-time **~60-second reload** if Ollama had unloaded the model. Four design decisions follow directly from that constraint:

1. **Decouple control cadence from the physics timestep.** EnergyPlus steps every 10 minutes internally, but the agent is only consulted every 2 simulated hours (`CONTROL_EVERY_N_HOURS` in `closed_loop_main.py`). Consulting an LLM every timestep would turn a multi-day run into a multi-hour ordeal for no control benefit — HVAC setpoints don't need to change faster than that in practice.

2. `keep_alive: "30m"` **on every Ollama** `/api/chat` **call**, so the model stays resident in memory between decisions instead of reloading (paying the ~60s cost) each time.

3. **Two-tier timeout.** Each individual HTTP call to Ollama has a 90-second client timeout; the entire multi-round decision (up to 3 tool-calling rounds) is bounded by a 240-second ceiling enforced on the asyncio future itself (`fut.result(timeout=DECISION_TIMEOUT_S)`). Bounding both the part and the whole prevents one slow round from silently consuming the entire budget meant for retries.

4. **Fail into a safe, known state, never block.** If the agent errors, times out, or returns something unparsable, `ep_runner.py`'s callback catches the exception, keeps the *last known-good setpoints*, and logs the fallback with its cause (`source=fallback_error`) — the simulation is never blocked or crashed by a slow or broken model call. In the current committed run this triggered twice across 49 decisions (47 resolved live, 2 fell back), with zero crashes — the mechanism working as designed rather than a hypothetical safeguard.

---

5. Handling Lengthy Simulation Logs

EnergyPlus's native output (`.err`, `.eso`, `.mtr`, and the `-r`/readvars CSV) is verbose, file-based, and grows with simulation length — not something to feed an LLM's context window directly, and not something whose size should affect the agent's behavior at all.

- **The live loop never reads those files.** It reads exactly the sensor and meter values it needs directly from memory through the EnergyPlus Python API's Data Exchange interface (`get_variable_value` / `get_meter_value`, via handles resolved once at startup), so what reaches the LLM is always a small, fixed-size JSON snapshot — a handful of per-zone values, outdoor temperature, power draw, and current setpoints — regardless of whether the simulation is on hour 1 or hour 96. The agent's context size is constant, not proportional to run length.
- **Two audiences, two log tiers.** `logs/live_state.json` is a small rolling snapshot meant only for the agent/MCP tools, overwritten each control step. `logs/ai_decisions.csv`, `logs/ai_run.csv`, and `logs/baseline_run.csv` are the durable, human- and dashboard-facing records — built incrementally in memory during the run and flushed to disk once at the end, never re-parsed from EnergyPlus's raw output.
- **Authoritative numbers come from EnergyPlus itself, not the live API.** The Data Exchange API's `get_meter_value` returns an instantaneous per-call reading, not a guaranteed running total, so it's used only for the agent-facing "recent power draw" signal. Every number the dashboard reports as a result (kWh

totals, comfort compliance) instead comes from `Output:Meter` / `Output:Variable` objects declared in the IDF and exported via EnergyPlus's own `-r`/readvars hourly CSV — independent accounting, produced once at the end of the run.

- **Raw files are kept, but only for humans.** `.err`/`.eso`/`eplusout.csv` under `logs/ai_run_raw/` and `logs/baseline_raw/` are retained on disk purely for manual debugging; no code path in the agent loop parses them.

---

6. Summary

Concern	Mechanism
Tool-calling	Real MCP subprocess + `ClientSession`, 3 tools, ≤3 rounds/decision, file-based state handoff
Prompt engineering	Rule-based system prompt, fixed-format state turn, concrete numeric pre-conditioning rule, defensive arg coercion
Latency	Decoupled control cadence, `keep_alive`, per-call + per-decision timeouts, fail-safe fallback
Simulation logs	In-memory Data Exchange API (fixed-size context) + separate EnergyPlus-native CSV for authoritative results

Together these choices let a small, locally-hosted model drive a multi-day physics simulation end-to-end without ever stalling, crashing, or needing its context to grow with the run.